yes, combine all of this into one small C header with everytyhing

Below is a small self-contained C header that bundles the full set: WGS84 lon/lat ↔ Mercator, lon/lat ↔ screen x/y, longitude normalization, latitude clamping, and helpers for approximate meters-per-Mercator-unit and meters-per-pixel at a given latitude. The Mercator forward and inverse formulas are the standard spherical/Web Mercator ones, and the ground-resolution helper follows the usual latitude-dependent cosine relationship used in Web Mercator scale calculations.[1] [2] [3]

Header

```
#ifndef SIMPLE_MERCATOR_H
#define SIMPLE_MERCATOR_H

#include <math.h>

#ifdef __cplusplus
extern "C" {
#endif

/*
 * Small WGS84/Web-Mercator utility header
 *
 * Coordinate model:
 * - Geographic coordinates are longitude/latitude in degrees.
 * - Mercator coordinates are projected coordinates:
 *       x = scale * lon_rad
 *       y = scale * log(tan(pi/4 + lat_rad/2))
 * - Screen coordinates assume:
 *       origin at top-left,
 *       +x to the right,
 *       +y downward,
 *       map center positioned at the window center.
 *
 * Notes:
 * - This is spherical/Web Mercator style math.
 * - Mercator latitude is clamped to +/- 85.05112878 degrees.
 * - `mercator_scale` defines the size of the Mercator coordinate system.
 * - `pixels_per_mercator` defines viewport zoom.
 */

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif
```

```c
#define MERCATOR_MAX_LAT_DEG 85.05112878
#define WGS84_EARTH_RADIUS_M 6378137.0
#define WGS84_EARTH_CIRCUMFERENCE_M (2.0 * M_PI * WGS84_EARTH_RADIUS_M)

typedef struct
{
    double longitude_deg;
    double latitude_deg;
} sm_geo_point_t;

typedef struct
{
    double x;
    double y;
} sm_mercator_point_t;

typedef struct
{
    double x;
    double y;
} sm_screen_point_t;

/* -------------------------- basic helpers -------------------------- */

static inline double sm_deg_to_rad(double deg)
{
    return deg * (M_PI / 180.0);
}

static inline double sm_rad_to_deg(double rad)
{
    return rad * (180.0 / M_PI);
}

static inline double sm_clamp_latitude_deg(double latitude_deg)
{
    if (latitude_deg > MERCATOR_MAX_LAT_DEG)  return MERCATOR_MAX_LAT_DEG;
    if (latitude_deg < -MERCATOR_MAX_LAT_DEG) return -MERCATOR_MAX_LAT_DEG;
    return latitude_deg;
}

static inline double sm_normalize_longitude_deg(double longitude_deg)
{
    double x = fmod(longitude_deg + 180.0, 360.0);
    if (x < 0.0)
        x += 360.0;
    return x - 180.0;
}

/* Optional: wrap to [0, 360) instead of [-180, 180) */
static inline double sm_normalize_longitude_deg_360(double longitude_deg)
{
    double x = fmod(longitude_deg, 360.0);
    if (x < 0.0)
        x += 360.0;
    return x;
```

```
}

/* -------------------------- mercator <-> geo -------------------------- */

static inline sm_mercator_point_t
sm_wgs84_to_mercator(double mercator_scale,
                     double longitude_deg,
                     double latitude_deg)
{
    double lon_rad = sm_deg_to_rad(longitude_deg);
    double lat_rad = sm_deg_to_rad(sm_clamp_latitude_deg(latitude_deg));

    sm_mercator_point_t p;
    p.x = mercator_scale * lon_rad;
    p.y = mercator_scale * log(tan(M_PI / 4.0 + lat_rad / 2.0));
    return p;
}

static inline sm_geo_point_t
sm_mercator_to_wgs84(double mercator_scale,
                     double mercator_x,
                     double mercator_y)
{
    sm_geo_point_t g;

    double lon_rad = mercator_x / mercator_scale;
    double lat_rad = 2.0 * atan(exp(mercator_y / mercator_scale)) - (M_PI / 2.0);

    g.longitude_deg = sm_rad_to_deg(lon_rad);
    g.latitude_deg  = sm_rad_to_deg(lat_rad);

    g.longitude_deg = sm_normalize_longitude_deg(g.longitude_deg);
    g.latitude_deg  = sm_clamp_latitude_deg(g.latitude_deg);

    return g;
}

/* -------------------------- geo <-> screen -------------------------- */

static inline sm_screen_point_t
sm_latlon_to_screen_xy(double mercator_scale,
                       double longitude_deg,
                       double latitude_deg,
                       double center_longitude_deg,
                       double center_latitude_deg,
                       double pixels_per_mercator,
                       int window_width,
                       int window_height)
{
    sm_mercator_point_t p = sm_wgs84_to_mercator(
        mercator_scale, longitude_deg, latitude_deg);

    sm_mercator_point_t c = sm_wgs84_to_mercator(
        mercator_scale, center_longitude_deg, center_latitude_deg);

    sm_screen_point_t s;
```

```
    s.x = (window_width  * 0.5) + (p.x - c.x) * pixels_per_mercator;
    s.y = (window_height * 0.5) - (p.y - c.y) * pixels_per_mercator;

    return s;
}

static inline sm_geo_point_t
sm_screen_xy_to_latlon(double mercator_scale,
                       double screen_x,
                       double screen_y,
                       double center_longitude_deg,
                       double center_latitude_deg,
                       double pixels_per_mercator,
                       int window_width,
                       int window_height)
{
    sm_mercator_point_t c = sm_wgs84_to_mercator(
        mercator_scale, center_longitude_deg, center_latitude_deg);

    sm_mercator_point_t p;
    p.x = c.x + (screen_x - window_width  * 0.5) / pixels_per_mercator;
    p.y = c.y - (screen_y - window_height * 0.5) / pixels_per_mercator;

    return sm_mercator_to_wgs84(mercator_scale, p.x, p.y);
}

/* -------------------------- mercator <-> screen -------------------------- */

static inline sm_screen_point_t
sm_mercator_to_screen_xy(double mercator_x,
                         double mercator_y,
                         double center_mercator_x,
                         double center_mercator_y,
                         double pixels_per_mercator,
                         int window_width,
                         int window_height)
{
    sm_screen_point_t s;
    s.x = (window_width  * 0.5) + (mercator_x - center_mercator_x) * pixels_per_merc
    s.y = (window_height * 0.5) - (mercator_y - center_mercator_y) * pixels_per_merc
    return s;
}

static inline sm_mercator_point_t
sm_screen_xy_to_mercator(double screen_x,
                         double screen_y,
                         double center_mercator_x,
                         double center_mercator_y,
                         double pixels_per_mercator,
                         int window_width,
                         int window_height)
{
    sm_mercator_point_t p;
    p.x = center_mercator_x + (screen_x - window_width  * 0.5) / pixels_per_mercator
    p.y = center_mercator_y - (screen_y - window_height * 0.5) / pixels_per_mercator
    return p;
```

```
}

/* -------------------------- distance / scale helpers -------------------------- */

/*
 * Approximate meters represented by 1 degree longitude at latitude.
 * Useful as a sanity check, not as a Mercator-space metric.
 */
static inline double sm_meters_per_degree_longitude(double latitude_deg)
{
    double lat_rad = sm_deg_to_rad(sm_clamp_latitude_deg(latitude_deg));
    return (WGS84_EARTH_CIRCUMFERENCE_M * cos(lat_rad)) / 360.0;
}

/*
 * Approximate meters represented by 1 degree latitude.
 * Good rough Earth-scale approximation.
 */
static inline double sm_meters_per_degree_latitude(void)
{
    return WGS84_EARTH_CIRCUMFERENCE_M / 360.0;
}

/*
 * Approximate meters per 1 Mercator unit at the given latitude.
 *
 * Since:
 *   mercator_x = mercator_scale * lon_rad
 * and one radian at latitude phi is R * cos(phi) meters on the ground,
 * then:
 *   meters_per_mercator_unit ~= (R * cos(phi)) / mercator_scale
 *
 * This is a local east-west ground scale. In Web Mercator, local scale
 * distortion is isotropic, so this is also the local meters-per-unit
 * factor for small distances around that latitude.
 */
static inline double sm_meters_per_mercator_unit(double mercator_scale,
                                                 double latitude_deg)
{
    double lat_rad = sm_deg_to_rad(sm_clamp_latitude_deg(latitude_deg));
    return (WGS84_EARTH_RADIUS_M * cos(lat_rad)) / mercator_scale;
}

/*
 * Approximate Mercator units per meter at the given latitude.
 */
static inline double sm_mercator_units_per_meter(double mercator_scale,
                                                 double latitude_deg)
{
    double mpu = sm_meters_per_mercator_unit(mercator_scale, latitude_deg);
    return (mpu != 0.0) ? (1.0 / mpu) : 0.0;
}

/*
 * Approximate meters per screen pixel at the given latitude.
 *
```

```
 * If one Mercator unit corresponds locally to:
 *   sm_meters_per_mercator_unit(...)
 * and you draw:
 *   pixels_per_mercator pixels per Mercator unit
 * then:
 *   meters_per_pixel = meters_per_mercator_unit / pixels_per_mercator
 */
static inline double sm_meters_per_pixel(double mercator_scale,
                                         double pixels_per_mercator,
                                         double latitude_deg)
{
    if (pixels_per_mercator == 0.0)
        return 0.0;

    return sm_meters_per_mercator_unit(mercator_scale, latitude_deg) /
           pixels_per_mercator;
}

/*
 * Approximate screen pixels per meter at the given latitude.
 */
static inline double sm_pixels_per_meter(double mercator_scale,
                                         double pixels_per_mercator,
                                         double latitude_deg)
{
    double mpp = sm_meters_per_pixel(mercator_scale, pixels_per_mercator, latitude
    return (mpp != 0.0) ? (1.0 / mpp) : 0.0;
}

/* -------------------------- visibility helpers -------------------------- */

static inline int sm_screen_point_is_inside(sm_screen_point_t p,
                                            int window_width,
                                            int window_height)
{
    return (p.x >= 0.0 && p.x < (double)window_width &&
            p.y >= 0.0 && p.y < (double)window_height);
}

#ifdef __cplusplus
}
#endif

#endif /* SIMPLE_MERCATOR_H */
```

The forward and inverse geographic transforms use the standard Mercator equations, while the screen conversion is the same center-based viewport mapping discussed earlier: translate relative to map center, scale by pixels-per-Mercator-unit, and invert Y for top-left-origin screens.[4] [2] [3]

Minimal example

```
#include <stdio.h>
#include "simple_mercator.h"

int main(void)
{
    double mercator_scale = WGS84_EARTH_RADIUS_M;
    double pixels_per_mercator = 0.002;

    double center_lon = 19.0000;
    double center_lat = 49.8000;

    int w = 1280;
    int h = 720;

    sm_screen_point_t s = sm_latlon_to_screen_xy(
        mercator_scale,
        19.0402, 49.8224,
        center_lon, center_lat,
        pixels_per_mercator,
        w, h
    );

    printf("screen: x=%f y=%f\n", s.x, s.y);

    sm_geo_point_t g = sm_screen_xy_to_latlon(
        mercator_scale,
        s.x, s.y,
        center_lon, center_lat,
        pixels_per_mercator,
        w, h
    );

    printf("geo: lon=%f lat=%f\n", g.longitude_deg, g.latitude_deg);

    printf("meters/pixel near center: %f\n",
           sm_meters_per_pixel(mercator_scale, pixels_per_mercator, center_lat));

    return 0;
}
```

If you set `mercator_scale = WGS84_EARTH_RADIUS_M`, then your Mercator X/Y are in the common "meters-like" Web Mercator form $x = R\lambda$, $y = R\ln(\tan(\pi/4 + \phi/2))$, and the local meters-per-pixel helper becomes easy to interpret at a chosen latitude.[5] [1]

Notes

`sm_meters_per_mercator_unit()` and `sm_meters_per_pixel()` are local approximations, not globally exact distances, because Mercator distorts scale with latitude; the cosine term captures that latitude dependence in the usual Web Mercator ground-resolution formula.[6] [1]

If you want, I can also give you a second version of this header that uses:

- explicit pan offsets instead of center lon/lat,
- a bounding-box viewport,
- or float-only types for embedded use.

⁂

1. https://gist.github.com/fgcarto/a24d89067f15551c622c
2. https://en.wikipedia.org/wiki/Mercator_projection
3. https://epsg.io/1026-method
4. https://stackoverflow.com/questions/47498309/inverting-web-mercator-equations-to-convert-pixels-to-latitude-longitude
5. https://proj.org/en/stable/operations/projections/merc.html
6. https://gist.github.com/perrygeo/4478844
7. https://copyprogramming.com/howto/inverting-web-mercator-equations-to-convert-pixels-to-latitude-longitude
8. https://stackoverflow.com/questions/30326797/math-behind-mkmeterspermappointatlatitude
9. https://visgl.github.io/math.gl/docs/modules/web-mercator/api-reference/web-mercator-utils
10. https://api.visicom.ua/en/products/tiles/theoretical-info
11. https://www.php.cn/faq/1796674483.html
12. https://stackoverflow.com/questions/14329691/convert-latitude-longitude-point-to-a-pixels-x-y-on-mercator-projection
13. https://gist.github.com/skyebook/3152659